

Digital IC Design

CSS PHY TRANSMITTER Project

Prepared for :

Dr. Ahmed Shalaby

Eng. Youssef Sherif

Eng. Ahmed Elgammal

Table of Contents

Abstract	4
Chapter1 – Introduction	5
1.1 Motivation	5
1.2 Problem Statement and Objectives	5
1.3 Scope	5
1.4 GitHub Link	5
Chapter 2 —System Architecture & Technical Specifications	6
2.1 IEEE 802.15.4 CSS PPDU Frame Structure	6
2.2 Transmitter Processing Pipeline	6
2.2.1 Zero-Padding & Demux (I/Q Splitter)	7
2.2.2 Symbol Mapper (Walsh–Hadamard Encoding)	7
2.2.3 Interleaver	7
2.2.4 PPDU Framer & Controller	7
2.2.5 QPSK Mapper & Differential QPSK (DQPSK) Encoder	7
2.2.6 Chirp Spread Keying (CSK) Generator & Modulator	7
2.3 System Interface & Pinout Description	8
Chapter 3: Verilog Hardware Design & Implementation	9
3.1 Top-Level Architecture & Datapath Strategy	9
3.2 Sub-Module Microarchitecture & Signal Interfaces	9
3.2.1 Payload RAM (ram.v)	9
3.2.2 Zero-Padding Unit (zero_padding.v)	10
3.2.3 Demultiplexer Unit (demux.v)	10
3.2.4 Symbol Mapper (symbol_mapper.v)	11
3.2.5 Symbol Buffer (symbol_buffer.v)	11
3.2.6 Interleaver Unit (interleaver.v)	11
3.2.7 PPDU Framer & Controller (PPDU.v)	12
3.2.8 QPSK Mapper (qpsk_mapper.v)	12
3.2.9 Differential QPSK (DQPSK) Encoder (dqpsk.v)	13
3.2.10 CSK Generator (CSK.v)	13
3.2.11 Final Multiplier	14
Chapter 5: Simulation, Functional Verification & Results	15
5.1 Verification using QuestaSim	15
5.2 Verification using MATLAB	18
5.2.1 Methodology	18

5.2.2 Modification on MATLAB Files	18
5.2.3 Verilog TestBench	19
5.3 FPGA implementation	20

Abstract

This project presents the design and RTL implementation of an IEEE 802.15.4 Chirp Spread Spectrum (CSS) Physical Layer transmitter. The transmitter receives a payload from the MAC layer and processes it through the required PHY operations, including zero padding, I/Q demultiplexing, symbol mapping, QPSK mapping, DQPSK coding, and CSS chirp generation. The design is implemented using Verilog HDL and verified through simulation using ModelSim. The objective is to develop a modular digital transmitter architecture that follows the processing requirements of the CSS PHY.

Chapter1 – Introduction

1.1 Motivation

Low-rate wireless personal area networks (LR-WPANS) require low cost, low power, and long battery life. IEEE 802.15.4 defines the MAC and PHY layers for this purpose, including the Chirp Spread Spectrum (CSS) PHY at 2450 MHz. CSS offers strong robustness to multipath fading and interference, and supports longer range and mobility compared to the standard's O-QPSK PHY, but it is significantly more complex to implement in hardware due to its chirp-based modulation and differential coding.

1.2 Problem Statement and Objectives

A floating-point MATLAB model of the CSS PHY transmitter already existed, implementing the processing chain defined in the standard. This work converts that model into working hardware: a fixed-point representation is derived, a Verilog RTL implementation is designed module by module, the RTL is verified against the MATLAB model at both unit and integration level, and the design is implemented and verified on ZedBoard (Zynq-7000 XC7Z020) FPGA.

1.3 Scope

This work covers the CSS PHY transmitter only; the receiver and MAC integration are left as future work. The existing MATLAB model is used solely as a verification reference, not developed as part of this work.

1.4 GitHub Link

That GitHub Link have all the codes and testbenches and constraints files needed for this Zigbee transmitter project.

- [GitHub link](#)

Chapter 2 —System Architecture & Technical Specifications

2.1 IEEE 802.15.4 CSS PPDU Frame Structure

The Physical Protocol Data Unit (PPDU) for the IEEE 802.15.4 Chirp Spread Spectrum (CSS) physical layer consists of three main components: the Synchronization Header (SHR), the PHY Header (PHR), and the PHY Service Data Unit (PSDU) payload.

1. Synchronization Header (SHR): Contains the Preamble sequence and the Start-of-Frame Delimiter (SFD). These allow the receiver to perform bit sync, frame boundary detection, and channel estimation.

2. PHY Header (PHR): Encodes critical packet parameters, including the total length of the payload in bytes (up to 127 bytes).

3. PHY Service Data Unit (PSDU): The actual payload data stream transferred from the MAC layer to the PHY layer.

Data rate	SHR		PHR	PSDU
	Preamble	SFD		
1 Mbps	8 symbols	4 symbols	2 symbols	variable
250 kbps	20 symbols	4 symbols	8 symbols	variable

Figure 1: Format of the CSS PPDU

2.2 Transmitter Processing Pipeline

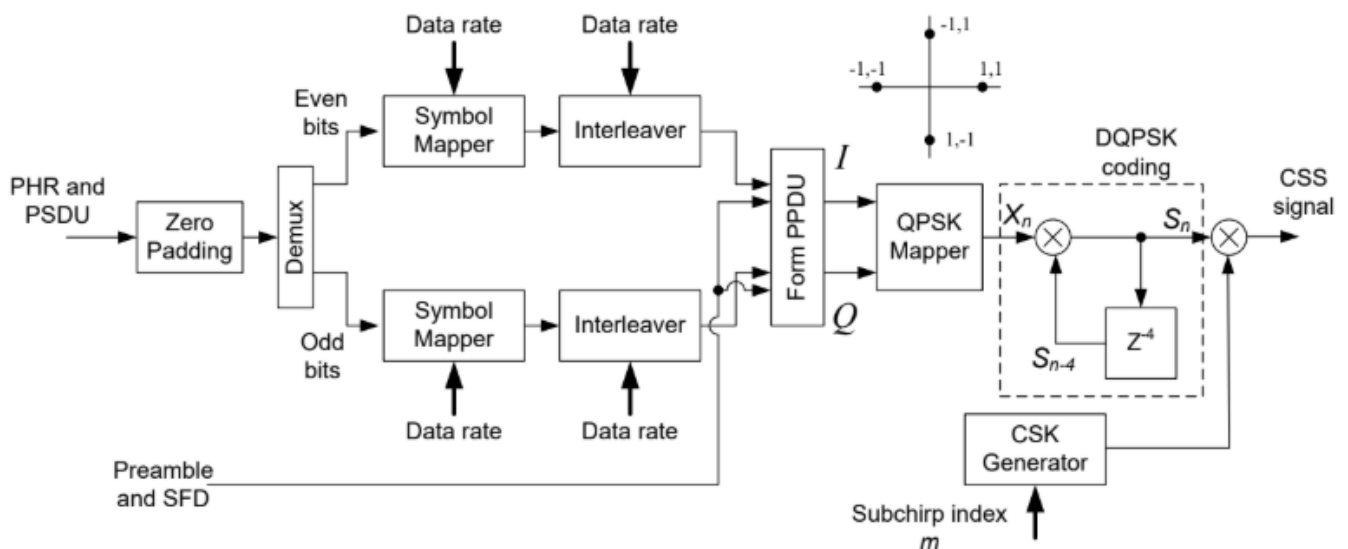


Figure 2: Transmitter Pipeline

The CSS physical layer transmitter processes the input data bit stream sequentially through multiple digital

processing blocks before outputting quadrature baseband signal samples.

2.2.1 Zero-Padding & Demux (I/Q Splitter)

Zero-Padding: Ensures that the bit stream comprising the PHR and PSDU is padded with trailing zeros so that its total length becomes a integer multiple of N ($N = 24$ for 250 kbps).

Demux: The serial padded bit stream is split into alternating even bits (in-phase I-channel) and odd bits (quadrature Q-channel).

2.2.2 Symbol Mapper (Walsh–Hadamard Encoding)

To enhance immunity against noise and interference, incoming groups of n bits are block-encoded into m -bit orthogonal Walsh–Hadamard codewords using predefined standard look-up tables (Tables 26a/26b of the IEEE 802.15.4 standard).

For 250 kbps operation, $n = 6$ input bits map to $m = 32$ codeword bits.

2.2.3 Interleaver

Operating exclusively in 250 kbps mode, the interleaver permutes groups of encoded bits to disperse burst errors. It reorders 16 groups of 4 bits each according to a fixed permutation matrix.

2.2.4 PPDU Framer & Controller

The controller coordinates the sequential assembly of the PPDU frame.

It inserts the static preamble and SFD patterns ahead of the PHR and encoded payload data streams before passing them down the datapath.

2.2.5 QPSK Mapper & Differential QPSK (DQPSK) Encoder

QPSK Mapper: Grouped pairs of (I, Q) bits are mapped onto standard QPSK constellation points.

DQPSK Encoder: To provide non-coherent phase modulation, each QPSK symbol is differentially encoded across 4 symbol periods:

$$S_n = X_n \cdot S_{n-4}$$

where S_n represents the complex differential symbol, X_n is the current QPSK symbol, and $S_0 = S_1 = S_2 = S_3 = \exp(j\pi/4)$ serve as the initialization vector.

2.2.6 Chirp Spread Keying (CSK) Generator & Modulator

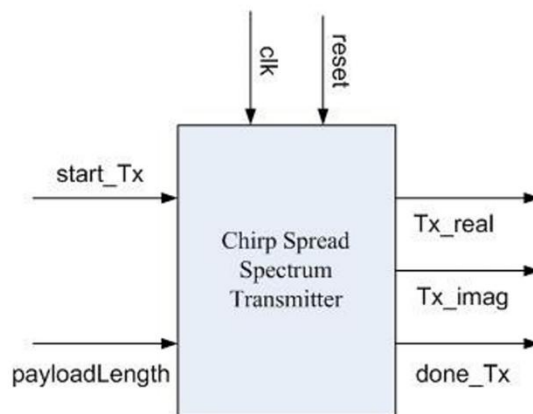
The primary DSP element of the transmitter: four successive DQPSK symbols select one of four possible 152-sample Chirp Sequences ($m \in \{1,2,3,4\}$).

Each chirp sequence is constructed from four 38-sample sinusoidal subchirps at a sampling frequency $f_s = 32$ MHz, shaped by a raised-cosine window (roll-off factor $\alpha = 0.25$).

The complex product of the DQPSK symbol sequence and the baseband Chirp Sequence yields the final complex output samples.

2.3 System Interface & Pinout Description

The top-level synthesizable Verilog module exposes a standardized interface designed for seamless integration with a MAC controller and system memory.



Signal name	Bit width	Direction	Type
<i>clk</i>	1	Input	Clock
<i>reset</i>	1	Input	Reset
<i>Start Tx</i>	1	Input	Control
<i>payloadlength</i>	8	Input	Data
<i>Payload din</i>	8	Input	Data
<i>Payload_addr</i>	7	Input	Address
<i>done tx</i>	1	Output	Control
<i>Tx_real</i>	8	Output	Data
<i>Tx_imag</i>	8	Output	data

Chapter 3: Verilog Hardware Design & Implementation

3.1 Top-Level Architecture & Datapath Strategy

The CSS PHY transmitter is implemented using a modular top-down architecture, separating control and datapath logic. The top-level module (*css_phy_tx*) orchestrates the pipeline stages through a centralized Finite State Machine (FSM), controlling signal flow from the initial payload configuration to the final baseband sample output.

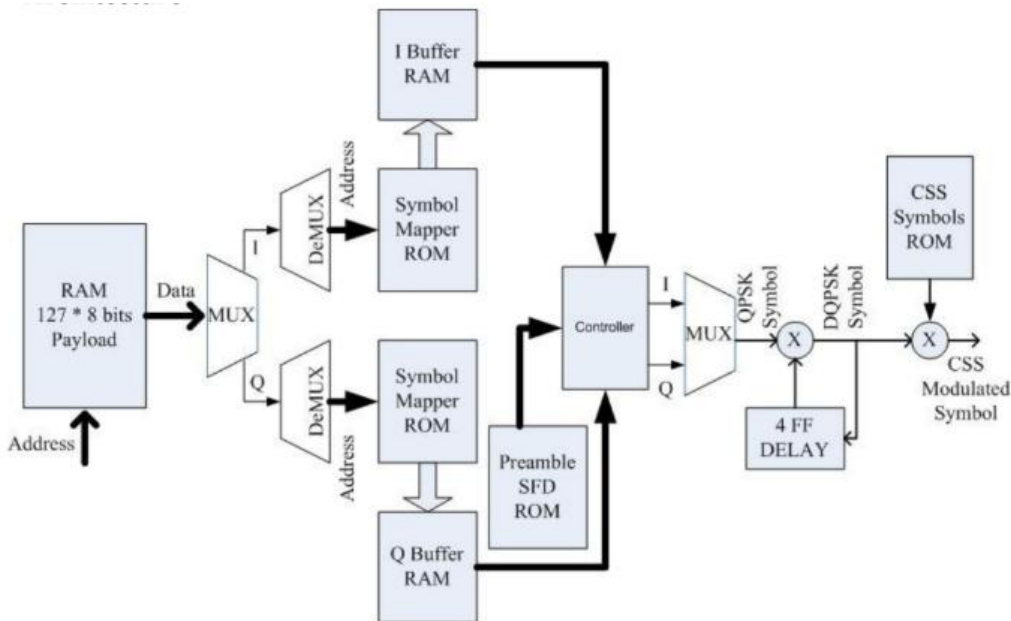


Figure 3: Transmitter Architecture

3.2 Sub-Module Microarchitecture & Signal Interfaces

3.2.1 Payload RAM (ram.v)

Microarchitecture & Logic: Implements a single-clock dual-port RAM storing up to 127 bytes (127×8 bits) of incoming MAC payload data. Data is written via *din* on assertion of *write_enable* and read out through *dout*. A generate block continuously exposes the internal memory array as a concatenated 1016-bit vector (*psdu_out*) to facilitate parallel frame assembly downstream.

Signal Name	Bit Width	Direction	Type	Description
<i>clk</i>	1	Input	Clock	System clock signal
<i>reset</i>	1	Input	Reset	Synchronous reset signal
<i>write_enable</i>	1	Input	Control	Write enable strobe for loading payload bytes
<i>read_enable</i>	1	Input	Control	Read enable strobe for reading byte output
<i>addr</i>	7	Input	Address	RAM byte address pointer (0 to 127)
<i>din</i>	8	Input	Data	Input data byte bus
<i>dout</i>	8	Output	Data	Registered output data byte bus
<i>psdu_out</i>	1016	Output	Data	Parallel 1016-bit concatenated payload bitstream bus.

3.2.2 Zero-Padding Unit (zero_padding.v)

Microarchitecture & Logic: Prepends the 12-bit PHY Header (PHR) constructed from payload length ahead of PSDU. It calculates required trailing zero-padding bits dynamically based on configured Data_Rate (250 kbps) to ensure total frame length is aligned to N-bit boundaries ($N = 24$). It outputs the serial frame bit by bit along with a complete padded_data vector.

Signal Name	Bit Width	Direction	Type	Description
<i>clk</i>	1	Input	Clock	System clock signal
<i>reset</i>	1	Input	Reset	Synchronous reset signal
<i>start_tx</i>	1	Input	Control	Frame transmission trigger pulse
<i>payload_length</i>	8	Input	Data	Total payload length in bytes
<i>PSDU</i>	1016	Input	Data	1016-bit full payload stream from Payload RAM
<i>out_bit</i>	1	Output	Data	Serial output bit stream
<i>valid_out</i>	1	Output	Control	Active-high output valid indicator
<i>padding_done</i>	1	Output	Control	Pulses high when total frame construction completes
<i>padded_data</i>	1032	Output	Data	Complete framed and zero-padded bitstream array
<i>total_bits</i>	11	Output	Data	Complete framed and zero-padded bitstream array

3.2.3 Demultiplexer Unit (demux.v)

Microarchitecture & Logic: Splits the continuous padded_data stream into parallel in-phase (*in_phase*, even bits) and quadrature (*quadrature*, odd bits) streams. A running control flag tracks processing state and increments bit indices by 2 each clock cycle until *total_bits* - 2 is reached.

Signal Name	Bit Width	Direction	Type	Description
<i>clk</i>	1	Input	Clock	System clock signal
<i>reset</i>	1	Input	Reset	Synchronous reset signal
<i>start_demux</i>	1	Input	Control	Demux processing trigger pulse
<i>padded_data</i>	1032	Input	Data	Zero-padded input frame array
<i>total_bits</i>	11	Input	Data	Total bit count of frame
<i>in_phase</i>	1	Output	Data	Even-indexed bit stream (I-path)
<i>quadrature</i>	1	Output	Data	Odd-indexed bit stream (Q-path)
<i>valid_out</i>	1	Output	Control	High when output I/Q bits are valid

3.2.4 Symbol Mapper (symbol_mapper.v)

Microarchitecture & Logic: Stores the normative IEEE 802.15.4 Walsh–Hadamard orthogonal code table inside a 64-entry 32-bit ROM lookup array. Maps 6-bit incoming symbol indices (*data_in*) to corresponding 32-bit orthogonal codewords (*data_out*) with a 1-cycle registered latency.

Signal Name	Bit Width	Direction	Type	Description
<i>clk</i>	1	Input	Clock	System clock signal
<i>reset</i>	1	Input	Reset	Synchronous reset signal
<i>valid_in</i>	1	Input	Control	Active-high input valid strobe
<i>data_in</i>	6	Input	Data	6-bit input symbol vector
<i>data_out</i>	32	Output	Data	32-bit mapped Walsh–Hadamard codeword
<i>valid_out</i>	1	Output	Control	Synchronized output valid indicator

3.2.5 Symbol Buffer (symbol_buffer.v)

Microarchitecture & Logic: Pairs and concatenates successive 32-bit codewords into 64-bit blocks[ci. Uses an internal toggling flag to register the first (even) 32-bit codeword and combine it with the second (odd) incoming 32-bit codeword to drive *intrlv_data* alongside an *intrlv_valid*.

Signal Name	Bit Width	Direction	Type	Description
<i>clk</i>	1	Input	Clock	System clock signal
<i>reset</i>	1	Input	Reset	Synchronous reset signal
<i>sym_valid</i>	1	Input	Control	Valid pulse from Symbol Mapper
<i>sym_data</i>	32	Input	Data	32-bit codeword from Symbol Mapper
<i>intrlv_valid</i>	1	Output	Control	High when concatenated 64-bit block is ready
<i>intrlv_data</i>	64	Output	Data	64-bit concatenated output block

3.2.6 Interleaver Unit (interleaver.v)

Microarchitecture & Logic: Implements the static 250 kbps permutation matrix using a fully combinational Verilog generate loop. Reorders 16 4-bit nibble groups of the incoming 64-bit block (*block_in*) to yield the permuted output block (*block_out*) with zero clock delay.

Signal Name	Bit Width	Direction	Type	Description
<i>block_in</i>	64	Input	Data	Un-interleaved 64-bit input block
<i>block_out</i>	64	Output	Data	Permuted 64-bit output block

3.2.7 PPDU Framer & Controller (PPDU.v)

Microarchitecture & Logic: Coordinates PPDU assembly using an internal state machine (IDLE, PREAMBLE, SFD, WAIT, SEND, DONE_STATETransmits an 80-symbol Preamble and 16-symbol SFD pattern from ROM before reading symbol blocks out of an integrated 8-entry 64-bit FIFO (fifo_i, fifo_q) and shifting them out bit by bit.

Signal Name	Bit Width	Direction	Type	Description
<i>clk</i>	1	Input	Clock	System clock signal
<i>reset</i>	1	Input	Reset	Synchronous reset signal
<i>start</i>	1	Input	Control	Transmission trigger signal
<i>total_bits</i>	11	Input	Data	Frame length parameter from Zero-Padding unit
<i>i_path_data</i>	64	Input	Data	In-phase 64-bit data block
<i>q_path_data</i>	64	Input	Data	Quadrature 64-bit data block
<i>i_path_valid</i>	1	Input	Control	Valid indicator for I-path input
<i>q_path_valid</i>	1	Input	Control	Valid indicator for Q-path input
<i>ppdu_i_out</i>	1	Output	Data	Serial I-path PPDU bit stream
<i>ppdu_q_out</i>	1	Output	Data	Serial Q-path PPDU bit stream
<i>ppdu_valid</i>	1	Output	Control	High during valid frame bit streaming
<i>done</i>	1	Output	Control	Pulse high on packet completion

3.2.8 QPSK Mapper (qpsk_mapper.v)

- Microarchitecture & Logic:** Maps 2-bit (I, Q) pairs to signed fixed-point QPSK constellation coordinates. Maps input bits {inphase, quadrature} to signed values $X_n \in \{-1, 0, 1\}$ represented in 2'sd format.

Signal Name	Bit Width	Direction	Type	Description
<i>clk</i>	1	Input	Clock	System clock signal
<i>reset</i>	1	Input	Reset	Synchronous reset signal
<i>inphase</i>	1	Input	Control	Incoming I-channel bit
<i>quadrature</i>	1	Input	Data	Incoming Q-channel bit
<i>valid_in</i>	1	Input	Control	Valid signal from PPDU Framer
<i>xn_real</i>	2	Output	Data	Signed real QPSK symbol component
<i>xn_imag</i>	2	Output	Data	Signed imaginary QPSK symbol component
<i>valid_out</i>	1	Output	Control	Output valid indicator

3.2.9 Differential QPSK (DQPSK) Encoder (dqpsk.v)

Microarchitecture & Logic: Converts input rectangular QPSK symbols into 8-phase values (in $phase \in \{0,2,4,6\}$ representing multiples of 45°). Executes complex differential multiplication via phase addition modulo 8 over 4 circulating feedback lanes ($mem[0:3]$). Initializes phase memories to $\exp(j\pi/4)$ (phase index 1) upon assertion of `reset` or `pkt_start`.

Signal Name	Bit Width	Direction	Type	Description
<code>clk</code>	1	Input	Clock	System clock signal
<code>reset</code>	1	Input	Reset	Synchronous reset signal
<code>pkt_start</code>	1	Input	Control	Reload pulse at packet start
<code>valid_in</code>	1	Input	Control	Valid signal from QPSK mapper
<code>xn_real</code>	2	Input	Data	Signed real QPSK component
<code>xn_imag</code>	2	Input	Data	Signed imaginary QPSK component
<code>out_valid</code>	1	Output	Control	Active-high output valid indicator
<code>out_phase</code>	3	Output	Data	3-bit phase output index representing 0° to 315° in steps of 45°

3.2.10 CSK Generator (CSK.v)

- Microarchitecture & Logic:** Generates 152-sample Chirp Spread Spectrum baseband waveform sequences. Integrates a 8-bit timer module to step through ROM addresses (0 to 151). Reads pre-quantized 6-bit signed fixed-point real and imaginary chirp coefficients (`real_out`, `imag_out`) stored in external lookup text files (`chirp_r`, `chirp_i`)

Signal Name	Bit Width	Direction	Type	Description
<code>clk</code>	1	Input	Clock	System clock signal
<code>CSK_rst</code>	1	Input	Reset	Timer reset signal
<code>real_out</code>	6	Output	Data	6-bit signed fixed-point real output sample
<code>imag_out</code>	6	Output	Data	6-bit signed fixed-point imaginary output sample

3.2.11 Final Multiplier

Microarchitecture & Logic: This block performs complex multiplication between the differentially encoded QPSK phase symbol S_n and the complex baseband chirp sample provided by the CSK generator.

Let the complex DQPSK symbol be expressed as:

$$S_n = \text{Re}(S_n) + j \text{Im}(S_n)$$

The complex baseband output sample is formulated as:

$$\text{Tx_complex} = S_n \cdot (\text{csk_real} + j \cdot \text{csk_imag})$$

Expanding into real and imaginary components:

$$\text{tx_real} = \text{Re}(S_n) \cdot \text{csk_real} - \text{Im}(S_n) \cdot \text{csk_imag}$$

$$\text{tx_imag} = \text{Re}(S_n) \cdot \text{csk_imag} + \text{Im}(S_n) \cdot \text{csk_real}$$

Since sn_phase takes discrete phase values restricted to 45° , 135° , 225° , and 315° (corresponding to $S_n \in \{\pm 1 \pm j1\}$), the hardware optimization eliminates full multipliers. Instead, it evaluates sn_phase via a conditional routing multiplexer structure (signed additions/subtractions):

sn_phase (3-bit)	Phase Angle	Equivalent S_n	Real Output (tx_real)	Imaginary Output (tx_imag)
3'd1	45°	$+1+j1$	$\text{csk_real} - \text{csk_imag}$	$\text{csk_real} + \text{csk_imag}$
3'd3	135°	$-1+j1$	$-\text{csk_real} - \text{csk_imag}$	$\text{csk_real} - \text{csk_imag}$
3'd5	225°	$-1-j1$	$\text{csk_imag} - \text{csk_real}$	$-\text{csk_real} - \text{csk_imag}$
3'd7	315°	$+1-j1$	$\text{csk_real} + \text{csk_imag}$	$\text{csk_imag} - \text{csk_real}$

Chapter 5: Simulation, Functional Verification & Results

5.1 Verification using QuestaSim

Using payload length = 4, for simplicity for debugging at first

- **Zero padding check : sending data serial bit by bit every clock cycle**

```

Transcript
#
# PAYLOAD = 4 BYTES
# PHR = 000000000100
# PSDU BYTE [0] / (RAM[0]) = 10001010
# PSDU BYTE [1] / (RAM[1]) = 00101010
# PSDU BYTE [2] / (RAM[2]) = 10100001
# PSDU BYTE [3] / (RAM[3]) = 11110011
#
# =====
# TIME = 36 | BIT_COUNT = 0 | OUT_BIT = 0
# TIME = 46 | BIT_COUNT = 1 | OUT_BIT = 0
# TIME = 56 | BIT_COUNT = 2 | OUT_BIT = 1
# TIME = 66 | BIT_COUNT = 3 | OUT_BIT = 0
# TIME = 76 | BIT_COUNT = 4 | OUT_BIT = 0
# TIME = 86 | BIT_COUNT = 5 | OUT_BIT = 0
# TIME = 96 | BIT_COUNT = 6 | OUT_BIT = 0
# TIME = 106 | BIT_COUNT = 7 | OUT_BIT = 0
# TIME = 116 | BIT_COUNT = 8 | OUT_BIT = 0
# TIME = 126 | BIT_COUNT = 9 | OUT_BIT = 0
# TIME = 136 | BIT_COUNT = 10 | OUT_BIT = 0
# TIME = 146 | BIT_COUNT = 11 | OUT_BIT = 0
# TIME = 156 | BIT_COUNT = 12 | OUT_BIT = 1
# TIME = 166 | BIT_COUNT = 13 | OUT_BIT = 0
# TIME = 176 | BIT_COUNT = 14 | OUT_BIT = 0
# TIME = 186 | BIT_COUNT = 15 | OUT_BIT = 0
# TIME = 196 | BIT_COUNT = 16 | OUT_BIT = 1
# TIME = 206 | BIT_COUNT = 17 | OUT_BIT = 0
# TIME = 216 | BIT_COUNT = 18 | OUT_BIT = 1
# TIME = 226 | BIT_COUNT = 19 | OUT_BIT = 0
# TIME = 236 | BIT_COUNT = 20 | OUT_BIT = 0
# TIME = 246 | BIT_COUNT = 21 | OUT_BIT = 0
# TIME = 256 | BIT_COUNT = 22 | OUT_BIT = 1
# TIME = 266 | BIT_COUNT = 23 | OUT_BIT = 0
# TIME = 276 | BIT_COUNT = 24 | OUT_BIT = 1
# TIME = 286 | BIT_COUNT = 25 | OUT_BIT = 0
# TIME = 296 | BIT_COUNT = 26 | OUT_BIT = 1
#
# =====
# TIME = 306 | BIT_COUNT = 27 | OUT_BIT = 0
# TIME = 316 | BIT_COUNT = 28 | OUT_BIT = 1
# TIME = 326 | BIT_COUNT = 29 | OUT_BIT = 0
# TIME = 336 | BIT_COUNT = 30 | OUT_BIT = 1
# TIME = 346 | BIT_COUNT = 31 | OUT_BIT = 0
# TIME = 356 | BIT_COUNT = 32 | OUT_BIT = 0
# TIME = 366 | BIT_COUNT = 33 | OUT_BIT = 0
# TIME = 376 | BIT_COUNT = 34 | OUT_BIT = 0
# TIME = 386 | BIT_COUNT = 35 | OUT_BIT = 1
# TIME = 396 | BIT_COUNT = 36 | OUT_BIT = 1
# TIME = 406 | BIT_COUNT = 37 | OUT_BIT = 1
# TIME = 416 | BIT_COUNT = 38 | OUT_BIT = 1
# TIME = 426 | BIT_COUNT = 39 | OUT_BIT = 1
# TIME = 436 | BIT_COUNT = 40 | OUT_BIT = 0
# TIME = 446 | BIT_COUNT = 41 | OUT_BIT = 0
# TIME = 456 | BIT_COUNT = 42 | OUT_BIT = 1
# TIME = 466 | BIT_COUNT = 43 | OUT_BIT = 1
# TIME = 476 | BIT_COUNT = 44 | OUT_BIT = 0
# TIME = 486 | BIT_COUNT = 45 | OUT_BIT = 0
# TIME = 496 | BIT_COUNT = 46 | OUT_BIT = 0
# TIME = 506 | BIT_COUNT = 47 | OUT_BIT = 0
#
# =====
# TOTAL_BITS = 48
# PADDING_BITS = 4
# Zero Padding FINISHED
# =====

```

- **Demux check : Even bits goes to Inphase path and odd bits goes to Quadrature path until every path collect 6 bits to start symbol mapping**

```

# TIME = 535 | DEMUX_BIT_COUNT = 0 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 546 | DEMUX_BIT_COUNT = 2 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 556 | DEMUX_BIT_COUNT = 4 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 566 | DEMUX_BIT_COUNT = 6 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 576 | DEMUX_BIT_COUNT = 8 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 586 | DEMUX_BIT_COUNT = 10 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 596 | DEMUX_BIT_COUNT = 12 | IN_PHASE = 1 | QUADRATURE = 0

```

- **Symbol mapper check : When collected 6 bits from demux start to take these 6 bits and mapping it into 32 bit**

```

# TIME = 535 | DEMUX_BIT_COUNT = 0 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 546 | DEMUX_BIT_COUNT = 2 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 556 | DEMUX_BIT_COUNT = 4 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 566 | DEMUX_BIT_COUNT = 6 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 576 | DEMUX_BIT_COUNT = 8 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 586 | DEMUX_BIT_COUNT = 10 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 596 | DEMUX_BIT_COUNT = 12 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 605 | I_SYM_COUNT = 0 | 6-BIT_I_SYM = 010000 (16) | 32-BIT_CODEWORD = 11111111111111110000000000000000
# TIME = 605 | Q_SYM_COUNT = 0 | 6-BIT_Q_SYM = 000000 (0) | 32-BIT_CODEWORD = 11111111111111111111111111111111
# TIME = 606 | DEMUX_BIT_COUNT = 14 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 616 | DEMUX_BIT_COUNT = 16 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 626 | DEMUX_BIT_COUNT = 18 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 636 | DEMUX_BIT_COUNT = 20 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 646 | DEMUX_BIT_COUNT = 22 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 656 | DEMUX_BIT_COUNT = 24 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 666 | I_SYM_COUNT = 1 | 6-BIT_I_SYM = 101101 (45) | 32-BIT_CODEWORD = 0101101010100101010101010100101
# TIME = 666 | Q_SYM_COUNT = 1 | 6-BIT_Q_SYM = 000000 (0) | 32-BIT_CODEWORD = 11111111111111111111111111111111

```

- **Interleaver check : After collecting 32 bit from symbol mapper wait again till collecting another 32 bit to be 64 bit to start interleaver, which divides 64 bits into 16 group with each group containing 4 bits**

```
# TIME = 605 | I_SYM_COUNT = 0 | 6-BIT_I_SYM = 010000 (16) | 32-BIT_CODEWORD = 11111111111111110000000000000000
# TIME = 605 | Q_SYM_COUNT = 0 | 6-BIT_Q_SYM = 000000 (0) | 32-BIT_CODEWORD = 11111111111111111111111111111111
# TIME = 606 | DEMUX_BIT_COUNT = 14 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 616 | DEMUX_BIT_COUNT = 16 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 626 | DEMUX_BIT_COUNT = 18 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 636 | DEMUX_BIT_COUNT = 20 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 646 | DEMUX_BIT_COUNT = 22 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 656 | DEMUX_BIT_COUNT = 24 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 666 | I_SYM_COUNT = 1 | 6-BIT_I_SYM = 101101 (45) | 32-BIT_CODEWORD = 01011010101001010101101010100101
# TIME = 666 | Q_SYM_COUNT = 1 | 6-BIT_Q_SYM = 000000 (0) | 32-BIT_CODEWORD = 11111111111111111111111111111111
# TIME = 666 | DEMUX_BIT_COUNT = 26 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 676 | I_BLOCK_COUNT = 0 | 64-BIT_BUFFER = 111111111111100000000000000001010101010010101010101010101011 | 64-BIT_INTERLEAVED = 11110101110101000010100000101010100001010000010111101011
# TIME = 676 | Q_BLOCK_COUNT = 0 | 64-BIT_BUFFER = 111111111111111111111111111111111111111111111111111 | 64-BIT_INTERLEAVED = 111111111111111111111111111111111111111111111111111
# TIME = 676 | DEMUX_BIT_COUNT = 28 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 686 | DEMUX_BIT_COUNT = 30 | IN_PHASE = 1 | QUADRATURE = 0
# TIME = 696 | DEMUX_BIT_COUNT = 32 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 706 | DEMUX_BIT_COUNT = 34 | IN_PHASE = 0 | QUADRATURE = 1
# TIME = 716 | DEMUX_BIT_COUNT = 36 | IN_PHASE = 1 | QUADRATURE = 1
# TIME = 726 | I_SYM_COUNT = 2 | 6-BIT_I_SYM = 111100 (60) | 32-BIT_CODEWORD = 00001111111100001111000000001111
# TIME = 726 | Q_SYM_COUNT = 2 | 6-BIT_Q_SYM = 000001 (1) | 32-BIT_CODEWORD = 10101010101010101010101010101010
# TIME = 726 | DEMUX_BIT_COUNT = 38 | IN_PHASE = 1 | QUADRATURE = 1
# TIME = 736 | DEMUX_BIT_COUNT = 40 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 746 | DEMUX_BIT_COUNT = 42 | IN_PHASE = 1 | QUADRATURE = 1
# TIME = 756 | DEMUX_BIT_COUNT = 44 | IN_PHASE = 0 | QUADRATURE = 0
# TIME = 766 | DEMUX_BIT_COUNT = 46 | IN_PHASE = 0 | QUADRATURE = 0

# =====
# DEMUX FINISHED
# =====

#
# TIME = 786 | I_SYM_COUNT = 3 | 6-BIT_I_SYM = 110100 (52) | 32-BIT_CODEWORD = 00001111000011111111000011110000
# TIME = 786 | Q_SYM_COUNT = 3 | 6-BIT_Q_SYM = 110100 (52) | 32-BIT_CODEWORD = 00001111000011111111000011110000
#
# =====
# TOTAL I SYMBOLS = 4 | TOTAL Q SYMBOLS = 4
# SYMBOL MAPPER FINISHED
# =====

#
# TIME = 796 | I_BLOCK_COUNT = 1 | 64-BIT_BUFFER = 0000111111110000111100000001111100001111111100001110000 | 64-BIT_INTERLEAVED = 000000001111000011111111000011110000000000011111111111110000
# TIME = 796 | Q_BLOCK_COUNT = 1 | 64-BIT_BUFFER = 101010101010101010101010101000001111111100001110000 | 64-BIT_INTERLEAVED = 10100001010000010101111010111100001010000101011110101111010111010111010
#
# =====
# TOTAL I BLOCKS = 2 | TOTAL Q BLOCKS = 2
# BUFFER & INTERLEAVER FINISHED
# =====
```

*Now we start to send every bit by bit to PPDU $\rightarrow$ QPSK $\rightarrow$ DQPSK*CSK $\rightarrow$ CSS Signal*

- PPDU check :** Verify that the PPDU Framer correctly assembles and transmits the complete frame by serializing the Preamble, SFD, PHR, and PSDU data on the I and Q paths. For 250 kbps, Preamble = 80 bits (all ones), SFD = 16 bits, PHR = 12 bits, and PSDU = $8 \times \text{payloadLength}$. For this test case, payloadLength = 4, so PHR+PSDU = $12 + (8 \times 4) = 44$ bits, which Zero Padding rounds up to the next multiple of $N=24$, giving 48 bits total. Demux splits this into 24 bits per path (I and Q), and the Symbol Mapper block-codes each path at rate $6 \rightarrow 32$ (each 6-bit input symbol becomes a 32-bit) expanding 24 bits into 4 codewords $\times 32$ bits = 128 bits per path. gives the final chip count per path: $80 \text{ (Preamble)} + 16 \text{ (SFD)} + 128 \text{ (coded PHR+PSDU)} = \mathbf{224 \text{ chips}}$.

- **QPSK Mapper check :** Verify that the QPSK Mapper correctly converts each (I, Q) chip pair from the PPDU into one complex QPSK symbol X_n according to

$$X_n = \frac{(I + Q) - j(I - Q)}{2}$$

The four input combinations are mapped as

$\{I, Q\} = \{0, 0\}$	$X_n = -1 + 0j$
$\{I, Q\} = \{0, 1\}$	$X_n = 0 + 1j$
$\{I, Q\} = \{1, 0\}$	$X_n = 0 - 1j$
$\{I, Q\} = \{1, 1\}$	$X_n = 1 + 0j$

- **DQPSK check:** Verify that the DQPSK Encoder generates S_n using the current QPSK symbol X_n and the symbol delayed by four positions:

$$S_n = X_n \cdot S_{n-4}$$

- The first four symbols are initialized at 45° . The output phase is checked to ensure that S_n always has one of the four expected phases: 45° , 135° , 225° , or 315° throughout the transmission.
- **CSK check** : Verify that the CSK Generator correctly generates the real and imaginary chirp samples from the ROM. The output samples are checked against the reference CSK values for the complete

transmission.

- **Final output check (tx real, tx img):** Verify that S_n is correctly combined with the CSK samples to generate Tx_{real} and Tx_{imag} using:

$$Tx_{real} = \text{Re}(S_n) \text{csk}_{real} - \text{Im}(S_n) \text{csk}_{imag}$$

$$Tx_{imag} = \text{Re}(S_n) \text{csk}_{imag} + \text{Im}(S_n) \text{csk}_{real}$$

For simplicity using this table again

sn_phase (3-bit)	Phase Angle	Equivalent S_n	Real Output (tx_real)	Imaginary Output (tx_imag)
3'd1	45°	+1+j1	csk_real - csk_imag	csk_real + csk_imag
3'd3	135°	-1+j1	-csk_real - csk_imag	csk_real - csk_imag
3'd5	225°	-1-j1	csk_imag - csk_real	-csk_real - csk_imag
3'd7	315°	+1-j1	csk_real + csk_imag	csk_imag - csk_real

At PPDU, QPSK, DQPSK, TX_CHIP_COUNT = 0 (First Bit)

$\{I, Q\} = \{1, 1\} \rightarrow$ Mapped by QPSK into $\{1+0j\}$

$\rightarrow$ Mapped to DQPSK that Phase = 1 (45°)

CSK_REAL_OUT = 111110 = -2

CSK_IMAG_OUT = 111011 = -5

According to the equation of final output at phase = 45° ,

$$Tx_{real} = \text{csk}_{real} - \text{csk}_{imag} = -2 - 5 = -7$$

$$Tx_{imag} = \text{csk}_{real} + \text{csk}_{imag} = -2 + 5 = 3$$

Verified correctly till the end of transcript 

```
# TIME = 815 | PPDU_CHIP_COUNT = 0 | I_OUT = 1 | Q_OUT = 1
# TIME = 825 | QPSK_CHIP_COUNT = 0 | XN_REAL = 1 | XN_IMAG = 0
# TIME = 826 | PPDU_CHIP_COUNT = 1 | I_OUT = 1 | Q_OUT = 1
# TIME = 835 | CSK_CHIP_COUNT = 0 | CSK_REAL_OUT = 111110 | CSK_IMAG_OUT = 111011
# TIME = 835 | DQPSK_CHIP_COUNT = 0 | IN_PHASE_CHIP = 1 | MAPPED_IN_PHASE = 0 | OUT_PHASE = 1 (0x1)
# TIME = 836 | PPDU_CHIP_COUNT = 2 | I_OUT = 1 | Q_OUT = 1
# TIME = 836 | QPSK_CHIP_COUNT = 1 | XN_REAL = 1 | XN_IMAG = 0
# TIME = 845 | TX_COUNT = 0 | TX_REAL = 3 | TX_IMAG = -7
# TIME = 846 | QPSK_CHIP_COUNT = 2 | XN_REAL = 1 | XN_IMAG = 0
# TIME = 846 | PPDU_CHIP_COUNT = 3 | I_OUT = 1 | Q_OUT = 1
# TIME = 846 | DQPSK_CHIP_COUNT = 1 | IN_PHASE_CHIP = 1 | MAPPED_IN_PHASE = 0 | OUT_PHASE = 1 (0x1)
# TIME = 846 | CSK_CHIP_COUNT = 1 | CSK_REAL_OUT = 110101 | CSK_IMAG_OUT = 000000
# TIME = 856 | CSK_CHIP_COUNT = 2 | CSK_REAL_OUT = 111100 | CSK_IMAG_OUT = 010000
# TIME = 856 | DQPSK_CHIP_COUNT = 2 | IN_PHASE_CHIP = 1 | MAPPED_IN_PHASE = 0 | OUT_PHASE = 1 (0x1)
# TIME = 856 | PPDU_CHIP_COUNT = 4 | I_OUT = 1 | Q_OUT = 1
# TIME = 856 | QPSK_CHIP_COUNT = 3 | XN_REAL = 1 | XN_IMAG = 0
# TIME = 856 | TX_COUNT = 1 | TX_REAL = -11 | TX_IMAG = -11
# TIME = 866 | TX_COUNT = 2 | TX_REAL = -20 | TX_IMAG = 12
# TIME = 866 | QPSK_CHIP_COUNT = 4 | XN_REAL = 1 | XN_IMAG = 0
# TIME = 866 | PPDU_CHIP_COUNT = 5 | I_OUT = 1 | Q_OUT = 1
# TIME = 866 | DQPSK_CHIP_COUNT = 3 | IN_PHASE_CHIP = 1 | MAPPED_IN_PHASE = 0 | OUT_PHASE = 1 (0x1)
# TIME = 866 | CSK_CHIP_COUNT = 3 | CSK_REAL_OUT = 010100 | CSK_IMAG_OUT = 001010
# TIME = 876 | CSK_CHIP_COUNT = 4 | CSK_REAL_OUT = 010100 | CSK_IMAG_OUT = 101101
# TIME = 876 | DQPSK_CHIP_COUNT = 4 | IN_PHASE_CHIP = 1 | MAPPED_IN_PHASE = 0 | OUT_PHASE = 1 (0x1)
# TIME = 876 | PPDU_CHIP_COUNT = 6 | I_OUT = 1 | Q_OUT = 1
# TIME = 876 | QPSK_CHIP_COUNT = 5 | XN_REAL = 1 | XN_IMAG = 0
# TIME = 876 | TX_COUNT = 3 | TX_REAL = 10 | TX_IMAG = 30
# TIME = 886 | TX_COUNT = 4 | TX_REAL = 39 | TX_IMAG = 1
# TIME = 886 | QPSK_CHIP_COUNT = 6 | XN_REAL = 1 | XN_IMAG = 0
# TIME = 886 | PPDU_CHIP_COUNT = 7 | I_OUT = 1 | Q_OUT = 1
# TIME = 886 | DQPSK_CHIP_COUNT = 5 | IN_PHASE_CHIP = 1 | MAPPED_IN_PHASE = 0 | OUT_PHASE = 1 (0x1)
# TIME = 886 | CSK_CHIP_COUNT = 5 | CSK_REAL_OUT = 110101 | CSK_IMAG_OUT = 100011
# TIME = 896 | CSK_CHIP_COUNT = 6 | CSK_REAL_OUT = 100001 | CSK_IMAG_OUT = 111101
# TIME = 896 | DQPSK_CHIP_COUNT = 6 | IN_PHASE_CHIP = 1 | MAPPED_IN_PHASE = 0 | OUT_PHASE = 1 (0x1)
```

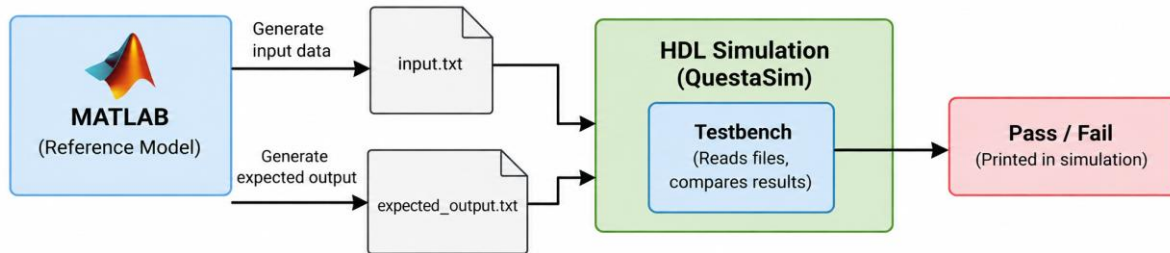
Transcript file

After sending 224 chirp all 224 chirp(as expected) sent successfully

```
#
# TOTAL PPDU CHIPS = 224
# FORM PPDU FINISHED
#
# =====
# QPSK MAPPER FINISHED (Total chips: 224)
# =====
#
# =====
# DQPSK ENCODER FINISHED (Total chips: 224)
# =====
#
# =====
# CSK GENERATOR FINISHED (Total loops monitored: 224)
# =====
#
# =====
# FINAL DQPSK OUTPUT FINISHED (Total chips: 224)
# =====
#
# =====
# ALL TESTS FINISHED SUCCESSFULLY
# =====
```

5.2 Verification using MATLAB

5.2.1 Methodology



The verification process starts with MATLAB, which acts as the reference model. MATLAB generates both the input test data and the expected output, saving them in separate .txt files. These files are then provided to the HDL design running in QuestaSim. During simulation, the testbench reads the input data and expected output directly from the files, applies the inputs to the design, and compares the generated HDL results with the MATLAB reference results. If all outputs match the expected values, the testbench reports **PASS**; otherwise, it reports **FAIL**. This process ensures that the HDL implementation behaves correctly and matches the MATLAB reference model.

5.2.2 Modification on MATLAB Files

- **Modifications Made to the MATLAB Files**

To prepare the MATLAB reference model for hardware implementation, the chirp sequence was converted from floating-point to a fixed-point representation compatible with the RTL design.

- **Chirp Sequence Quantization**

The chirp waveform is initially generated using complex exponential functions, producing floating-point values. These samples were quantized in runMe.m using the selected DAC resolution:

```
chirpSequence_Tx = floor(chirpSequence *(2^(TxDACbitNumber-1)-1));
```

With TxDACbitNumber = 6, the samples are scaled and truncated to integer values.

- **Fixed-Point Representation**

The quantized real and imaginary components were converted into MATLAB fixed-point objects using a signed 6-bit format with zero fractional bits:

```
re = fi(chirpSequenceReal_tofile, 1, 6, 0);
```

```
im = fi(chirpSequenceImag_tofile, 1, 6, 0);
```

The complex chirp sequence was then separated into real and imaginary components and converted into two's-complement binary strings.

- **ROM File Generation**

Finally, the binary values were written to separate text files:

```
chirpSequenceReal_tofile.txt
```

```
chirpSequenceImag_tofile.txt
```

These files are used to initialize the corresponding ROMs in the RTL design through \$readmemb.

- **Summary**

The main modification was quantizing the floating-point chirp waveform and converting it into a 6-bit signed fixed-point binary format suitable for hardware implementation.

The selected resolution of $R = 6$ bits was chosen because it provides sufficient accuracy to meet the required MSE specification while reducing hardware complexity, ROM size, and power consumption compared to using a higher resolution such as $R = 8$ bits.

5.2.3 Verilog TestBench

A self-checking SystemVerilog testbench was developed to verify the CSS PHY transmitter against the MATLAB reference model. MATLAB generates the input payload along with the expected real and imaginary output samples, which are stored in .txt files. During simulation, the testbench loads the input data, applies it to the DUT, and compares the generated transmitter outputs with the corresponding MATLAB reference samples on a sample-by-sample basis. Any mismatch is detected and reported, while a complete match produces a **PASS** result. The verification is performed for multiple payload sizes to ensure correct operation under different input conditions. The complete RTL design, testbench, and supporting files are available through the **GitHub link provided in Section 1.4**.

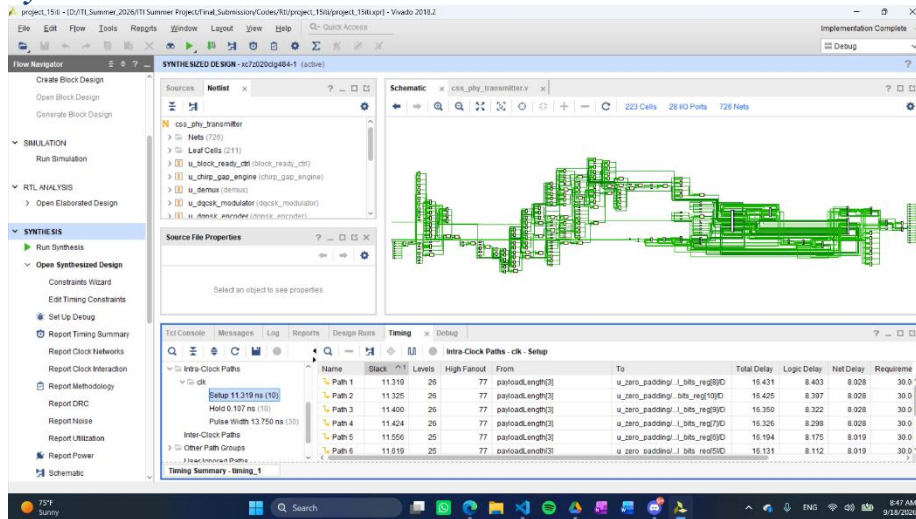
```
# =====
# RUNNING TEST CASE: Payload Length = 5 bytes (SYMBOL-RATE check)
# =====
# -----
# TEST CASE COMPLETED (Len = 5 bytes)
# Total Samples Checked = 13824 (chip-rate: 1 sample per RTL output)
# Total Errors = 0
# >>> RESULT: PASS <<<
# =====
#
# =====
# RUNNING TEST CASE: Payload Length = 20 bytes (SYMBOL-RATE check)
# =====
# -----
# TEST CASE COMPLETED (Len = 20 bytes)
# Total Samples Checked = 29184 (chip-rate: 1 sample per RTL output)
# Total Errors = 0
# >>> RESULT: PASS <<<
# =====
#
# =====
# RUNNING TEST CASE: Payload Length = 55 bytes (SYMBOL-RATE check)
# =====
# -----
# TEST CASE COMPLETED (Len = 55 bytes)
# Total Samples Checked = 62976 (chip-rate: 1 sample per RTL output)
# Total Errors = 0
# >>> RESULT: PASS <<<
# =====
#
# =====
# RUNNING TEST CASE: Payload Length = 125 bytes (SYMBOL-RATE check)
# =====
# -----
# TEST CASE COMPLETED (Len = 125 bytes)
# Total Samples Checked = 136704 (chip-rate: 1 sample per RTL output)
# Total Errors = 0
# >>> RESULT: PASS <<<
#
```

[Whole transcript](#)

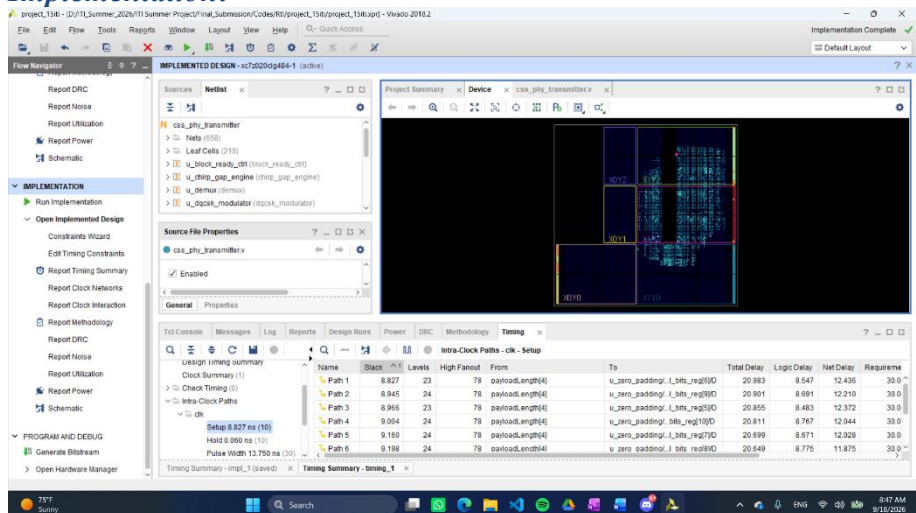
5.3 FPGA implementation

Using clock period = 30ns , as there was no constraint on operating frequency

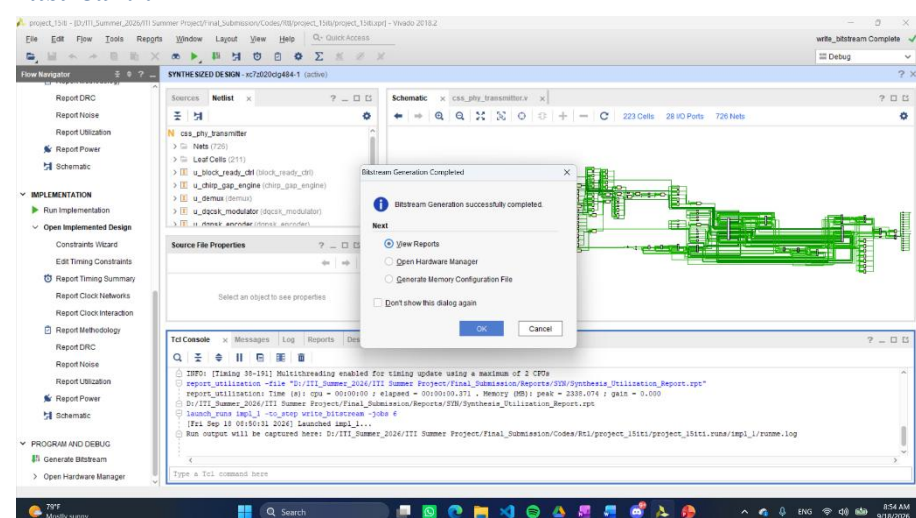
• Synthesis :



• Implementation:



• Bitstream :



Reports for timing :

- [Synthesis](#)
- [Implementation](#)

Reports for utilization:

- [Synthesis](#)
- [Implementation](#)